Hashmaps And Linked Lists

Hash Map Pattern

Definition:

Stores key-value pairs,
enabling fast data retrieval by directly accessing values through unique keys

Use Cases:

Fast data access and retrieval.
Storing relationships between pairs of data.
Implementing dictionaries, database indexing, caches.

Example:

HashMap
+Key1 : Value1 +Key2 : Value2 +Key3 : Value3

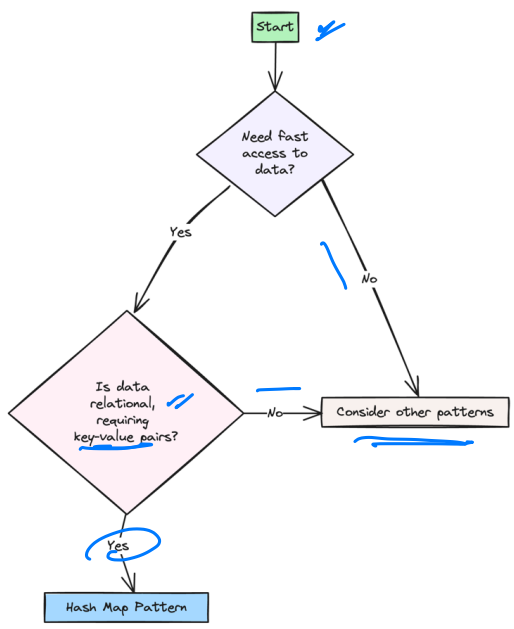
Advantages:

Efficient Data Access: O(1) time complexity for lookup, insert, and delete operations on average.

Flexibility in Key Usage: Unlike arrays, keys in hash maps can be of various data types

Avoids Data Duplication 0, 1, 2, . . .

Does My Problem Match This Pattern?




```
{
    key1: value1,
    key2: value2
    :
}
```

dictionaries

map = {} (or) dict( )

PROBLEM - 1

347. Top K Frequent Elements

Solved

Medium | Topics | Companies

Given an integer array `nums` and an integer `k`, return *the* `k` *most frequent elements*. You may return the answer in **any order**.

Example 1:

> **Input:** `nums = [1,1,1,2,2,3], k = 2`
>
> **Output:** `[1,2]`

Example 2:

> **Input:** `nums = [1], k = 1`
>
> **Output:** `[1]`

Example 3:

> **Input:** `nums = [1,2,1,2,1,2,3,1,3,2], k = 2`
>
> **Output:** `[1,2]`

Constraints:

- `1 <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`
- `k` is in the range `[1, the number of unique elements in the array]`.
- It is **guaranteed** that the answer is **unique**.

Brute

nums = [1, 1, 1, 2, 2, 3], K = 2

(0 1 2 3 4 5)

3 2 1

[1, 2]

{
1 : 3
2 : 2
3 : 1
}

[(1, 3), (2, 2), (3, 1)]

after sort

[(1, 3), (2, 2), (3, 1)]

decreasing

K = 2

T.C = O(n) + O(n log(n))

S.C = O(n)

Better: Heap

n = len. of nums

Best / Optimal: nums = [1, 1, 1, 2, 2, 3], K = 2

{
1 : 3,
2 : 2,
3 : 1
}

0	1	2	3	4	5	6	7

[3] [2] [1]

we for sure know the max-freq. we can encounter is "n"

T.C = O(n) + O(n)

S.C = O(n) + O(n)

nums = [1, 1, 1, 2, 2, 2, 3, 3, 4], K = 2

{
1 : 3,
2 : 3,
3 : 2,
4 : 1
}

0	1	2	3	4	5	6	7	8	9

size = n+1

[4] [3] [1, 2]

K = 2

[1, 2]

Linked Lists Patterns

Fast & Slow Pointers

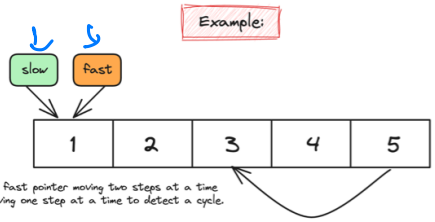
Definition:

uses two pointers traversing an iterable data structure at different speeds to identify distinguishable features

Use Cases:

- Detecting cycles in a linked list or sequence.
- Finding the middle element of a linked list.
- Determining if a list is palindromic.
- Finding the meeting point in a cycle.

Example:



In a linked list, use a fast pointer moving two steps at a time and a slow pointer moving one step at a time to detect a cycle.

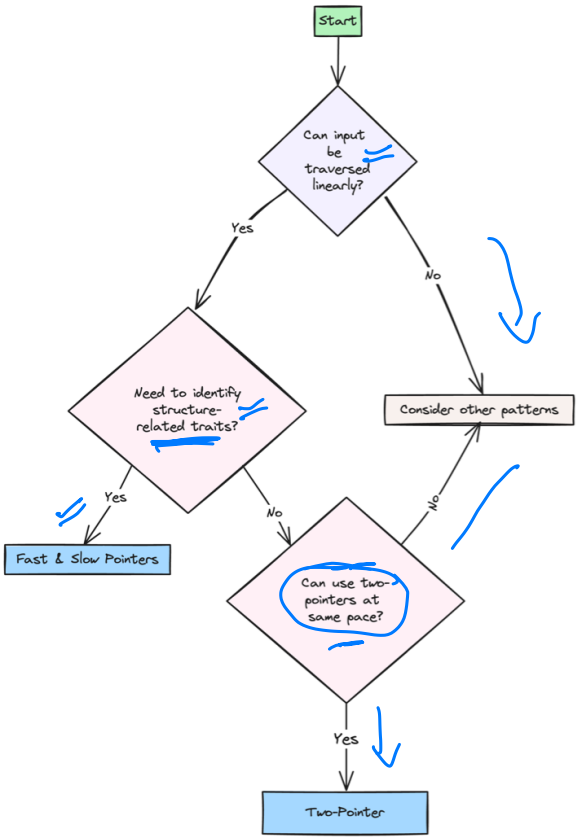
Advantages:

Efficient for cycle detection without extra space.

Useful for finding specific elements (like the middle) in a single pass.

Requires minimal extra space (O(1) space complexity)

Does My Problem Match This Pattern?



In-place Reversal of a Linked List

Definition:

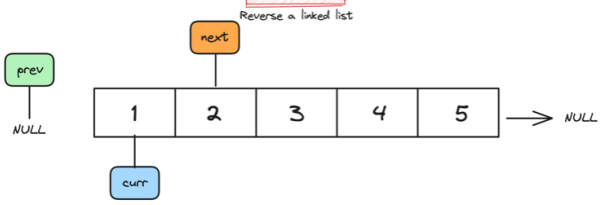
Reorganizes the nodes of a linked list in the opposite order directly, without additional memory.

Use Cases:

Reversing an entire singly or doubly linked list.
Reversing sub-lists within a linked list for pattern matching or reordering.
Used as a step in more complex algorithms, like palindrome checks.

Example:

Reverse a linked list

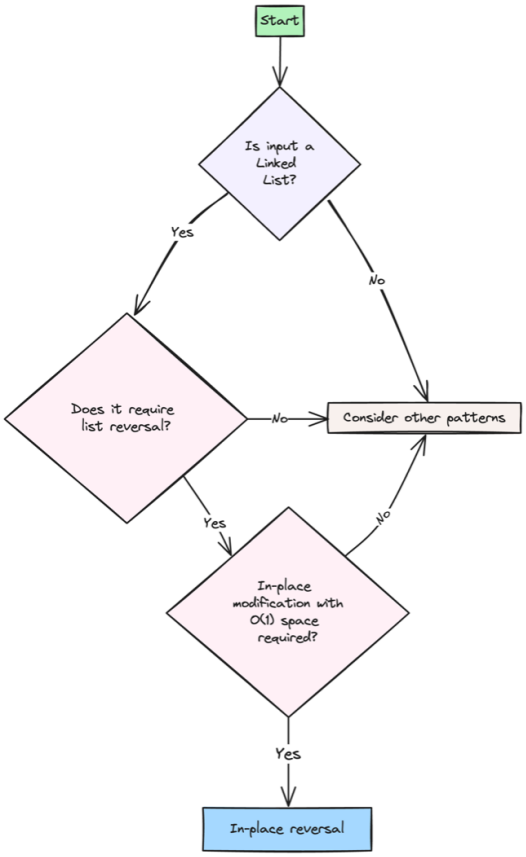


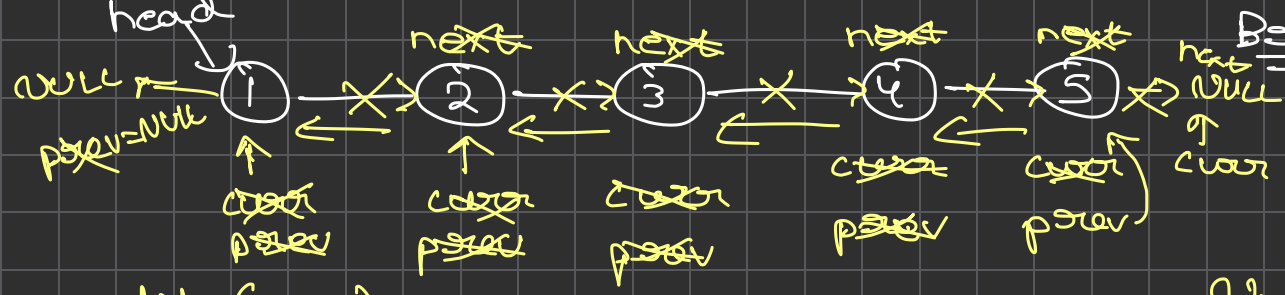
Advantages:

Space Efficiency: Utilizes O(1) space, not requiring extra lists or arrays.

Time Complexity: Achieves O(n) time efficiency by reorienting pointers in a single pass.

Does My Problem Match This Pattern?





Brute:

T.C: $O(2n)$

S.C: $O(n)$

```
while (curr)
{
    next = curr.next
    curr.next = prev

    prev = curr
    curr = next
}

return prev;
```

T.C = $O(n)$

S.C = $O(1)$

Node:

```
__init__(self):
    self.val = val
    self.next = None
```

PROBLEM- 2

2095. Delete the Middle Node of a Linked List

Solved

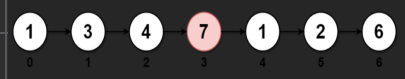
Medium | Topics | Companies | Hint

You are given the `head` of a linked list. **Delete** the **middle node**, and return *the* `head` *of the modified linked list*.

The **middle node** of a linked list of size `n` is the `⌊n / 2⌋th` node from the **start** using **0-based indexing**, where `⌊x⌋` denotes the largest integer less than or equal to `x`.

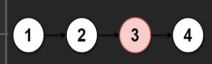
- For `n` = `1`, `2`, `3`, `4`, and `5`, the middle nodes are `0`, `1`, `1`, `2`, and `2`, respectively.

Example 1:



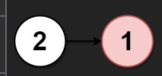

```
Input: head = [1,3,4,7,1,2,6]
Output: [1,3,4,1,2,6]
Explanation:
The above figure represents the given linked list. The indices of the nodes are written below.
Since n = 7, node 3 with value 7 is the middle node, which is marked in red.
We return the new list after removing this node.
```

Example 2:




```
Input: head = [1,2,3,4]
Output: [1,2,4]
Explanation:
The above figure represents the given linked list.
For n = 4, node 2 with value 3 is the middle node, which is marked in red.
```

Example 3:



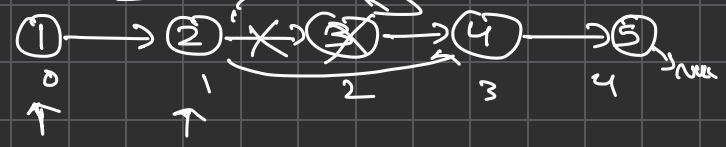

```
Input: head = [2,1]
Output: [2]
Explanation:
The above figure represents the given linked list.
For n = 2, node 1 with value 1 is the middle node, which is marked in red.
Node 0 with value 2 is the only node remaining after removing node 1.
```

Constraints:

- The number of nodes in the list is in the range `[1, 10^5]`.
- `1 <= Node.val <= 10^5`

Brute

odd



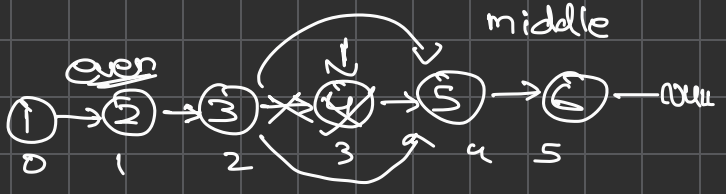
$5//2 = 2-1$

index == (n//2 - 1) !

curr.next = curr.next.next

middle

even



$6//2 = 3 - 1$

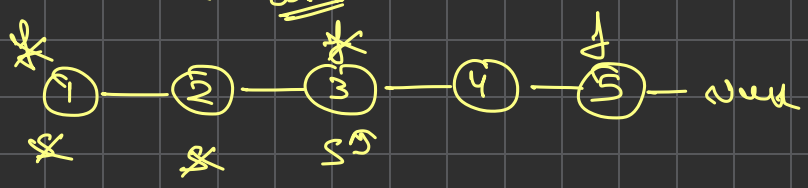
T.C = O(n) + O(n//2)

S.C = O(1)

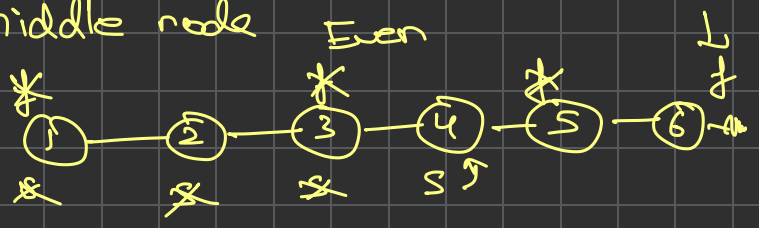
Optimal:

odd

How to find middle node

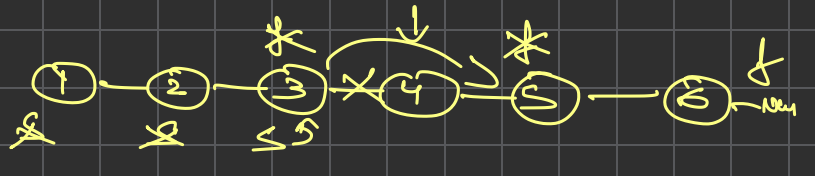


Even



T.C = O(n//2)
S.C = O(1)
} Complexities

Delete middle node



s.next = s.next.next

T.C = O(n//2)
S.C = O(1)

PROBLEM - 3

141. Linked List Cycle

Solved

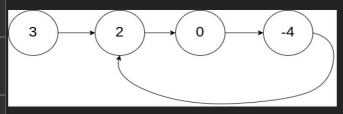
Easy | Topics | Companies

Given `head`, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to. **Note that `pos` is not passed as a parameter.**

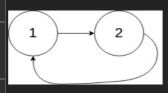
Return `true` *if there is a cycle in the linked list*. Otherwise, return `false`.

Example 1:




```
Input: head = [3,2,0,-4], pos = 1
Output: true
Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).
```

Example 2:



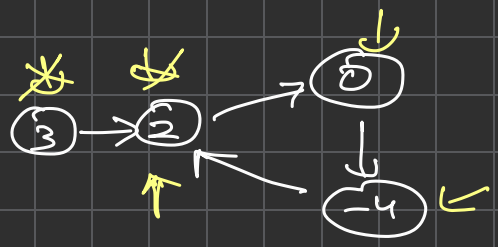

```
Input: head = [1,2], pos = 0
Output: true
Explanation: There is a cycle in the linked list, where the tail connects to the 0th node.
```

Example 3:

```
Input: head = [1], pos = -1
Output: false
Explanation: There is no cycle in the linked list.
```

Constraints:

- The number of the nodes in the list is in the range `[0, 10⁴]`.
- `-10⁵ <= Node.val <= 10⁵`
- `pos` is `-1` or a **valid index** in the linked-list.



Brute:

{ 3 : 1
2 : 1
0 : 1
-4 : 1 }

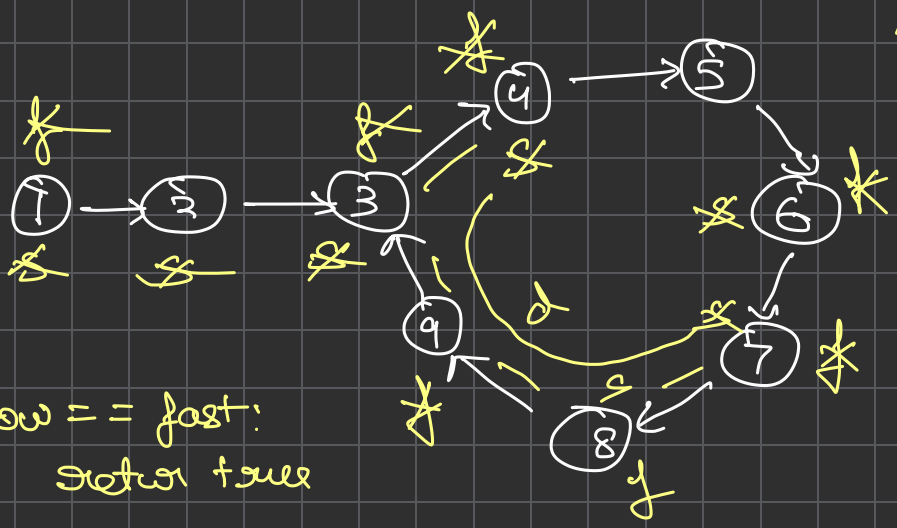
return true

{ 1 : 1
2 : 1
3 : 1 }

return false.

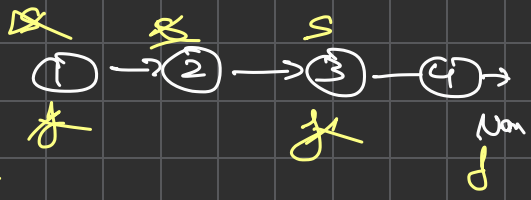
Optimal

T.C = O(n)
S.C = O(n)



slow == fast:
return true

return false



d = 4	d
d = 3	d − 1
d = 2	d − 2
d = 1	d − 3
d = 0	⋮
	d − d

PROBLEM - 4

49. Group Anagrams

Medium | Topics | Companies

Given an array of strings `strs`, group the anagrams together. You can return the answer in **any order**.

Example 1:

> **Input:** `strs = ["eat","tea","tan","ate","nat","bat"]`
>
> **Output:** `[["bat"],["nat","tan"],["ate","eat","tea"]]`
>
> **Explanation:**
>
> - There is no string in strs that can be rearranged to form `"bat"`.
> - The strings `"nat"` and `"tan"` are anagrams as they can be rearranged to form each other.
> - The strings `"ate"`, `"eat"`, and `"tea"` are anagrams as they can be rearranged to form each other.

Example 2:

> **Input:** `strs = [""]`
>
> **Output:** `[[""]]`

Example 3:

> **Input:** `strs = ["a"]`
>
> **Output:** `[["a"]]`

⇒ Anagram: It is a word or phrase formed by re-arranging the letters of a different word or phrase, using all the original chars. exactly once.

eat → aet
tea → aet
ate → aet

tan → ant
nat → ant

bat

k → avg. len. of words in list

T.C = O(n) × O(k log(k))
S.C = O(n)

a	b	c	d	e	f	...	z
0	1	2	3	4	5	...	25
0	0	0	0	0	0		0

T.C = O(n.k)
S.C = O(n + 26)

{
aet : [eat, tea, ate],
ant : [tan, nat]
abt : [bat]
}

(a – z) → 26 letters

strs = [eat, tea, tan, ate, nat, bat]

{
(----) : [eat, tea, ate],
(----) : [tan, nat]
(----) : [bat]
}